Q1. Computer Performance

A processor executes a program consisting of three instruction classes. The instruction counts and average CPI values are given below.

Instruction Class	Instruction Count	CPI
Arithmetic	400 million	1
Load/Store	300 million	3
Branch	100 million	2

The processor operates at a clock frequency of 2 GHz.

(a) Calculate the average CPI and the total CPU execution time of the program.

(b) A new processor design reduces the CPI of Load/Store instructions from 3 to 2, while all other parameters remain unchanged. Calculate the new CPU execution time and the overall speedup.

(c) Instead of improving Load/Store instructions, another design increases the clock frequency from 2 GHz to 2.5 GHz while keeping the original CPI values. Determine which design, (b) or (c), provides better performance and calculate the percentage difference in execution time relative to the slower design.

Answer 1

(a) Average CPI and CPU Execution Time

Total instruction count:

$$IC = 400 + 300 + 100 = 800 \text{ million}$$

Average CPI:

$$CPI_{avg} = \frac{400(1) + 300(3) + 100(2)}{800} = \boxed{1.875}$$

CPU execution time:

$$T_{CPU} = \frac{IC \times CPI_{avg}}{f}$$

$$= \frac{800 \times 10^6 \times 1.875}{2 \times 10^9} = \boxed{0.75 \text{ s}}$$

(b) Improved Load/Store CPI

New total clock cycles:

$$400(1) + 300(2) + 100(2) = 1200 \text{ million}$$

New CPU execution time:

$$T_b = \frac{1200 \times 10^6}{2 \times 10^9} = \boxed{0.60 \text{ s}}$$

Speedup:

$$Speedup = \frac{0.75}{0.60} = \boxed{1.25}$$

(c) Increased Clock Frequency

CPU execution time:

$$T_c = \frac{1500 \times 10^6}{2.5 \times 10^9} = \boxed{0.60 \text{ s}}$$

Percentage difference relative to the slower design:

$$\frac{|T_b - T_c|}{\max(T_b, T_c)} \times 100\% = \boxed{0\%}$$

Q2. MIPS Instructions and Machine Code

Consider the following MIPS instructions, starting at memory address 0x00400020.

Address	Instruction
0x00400020	`add $t0, $s1, $s2`
0x00400024	`lw $t1, 12($t0)`
0x00400028	`beq $t1, $zero, TARGET`
0x0040002C	`sub $t2, $t1, $s3`
0x00400030	`addi $t3, $t3, 1`
0x00400034	`TARGET: or $t4, $t0, $t2`

Assume standard 32-bit MIPS instruction encoding.

(a) Identify the instruction format (R-type, I-type, or J-type) of the add, lw, and beq instructions. For each instruction, specify the values of its encoding fields in decimal.

(b) Convert the following instruction into its 32-bit machine code in hexadecimal.

```
lw $t1, 12($t0)
```

(c) Calculate the 16-bit branch offset of the beq instruction. Express the offset in decimal and hexadecimal. If $t1 = 0, determine the next instruction address executed after the branch.

Answer 2

(a) Instruction Formats and Encoding Fields

1. add $t0, $s1, $s2

R-type instruction:

- opcode = 0
- rs = 17 ($s1)
- rt = 18 ($s2)
- rd = 8 ($t0)
- shamt = 0
- funct = 32

2. lw $t1, 12($t0)

I-type instruction:

- opcode = 35
- rs = 8 ($t0)
- rt = 9 ($t1)
- immediate = 12

3. beq $t1, $zero, TARGET

I-type instruction:

- opcode = 4
- rs = 9 ($t1)
- rt = 0 ($zero)
- immediate = 2

(b) Machine Code Encoding

Instruction:

lw $t1, 12($t0)

Binary encoding:

$$\underbrace{100011}_{opcode}\ \underbrace{01000}_{rs}\ \underbrace{01001}_{rt}\ \underbrace{0000000000001100}_{immediate}$$

32-bit hexadecimal machine code:

$$\boxed{0\text{x}8\text{D}09000\text{C}}$$

(c) Branch Offset and Target Address

Branch instruction address:

$$PC = 0\text{x}00400028, \qquad PC + 4 = 0\text{x}0040002\text{C}$$

Target address:

$$TARGET = 0\text{x}00400034$$

Branch offset:

$$Offset = \frac{TARGET-(PC+4)}{4} = \frac{0\text{x}00400034-0\text{x}0040002\text{C}}{4} = 2$$

16-bit hexadecimal offset:

$$\boxed{0\text{x}0002}$$

Since $t1 = 0, the branch condition is satisfied.

Therefore, the next instruction address is:

$$\boxed{0\text{x}00400034}$$

Q3. MIPS Program Analysis

```
        addi $t0, $zero, 0

        addi $t1, $zero, 4

        addi $t2, $zero, 0

LOOP:   lw   $t3, 0($a0)

        add  $t2, $t2, $t3

        addi $a0, $a0, 4

        addi $t0, $t0, 1

        bne  $t0, $t1, LOOP
        sw   $t2, 0($a1)
```

Assume the initial register values are:

- `$a0 = 0x10010000`
- `$a1 = 0x10020000`

Memory Address	**Value**
0x10010000	5
0x10010004	8
0x10010008	3
0x1001000C	6

Assume that branch delay slots are not used.

(a) Determine the number of iterations executed by the loop and the final values of registers $t0, $t2, and $a0.

(b) Determine the memory address and value written by the sw instruction. Also calculate the total number of dynamically executed instructions, including the final sw.

(c) Suppose the instruction addi $a0, $a0, 4 is changed to addi $a0, $a0, 8. Assume that memory locations 0x10010010 and 0x10010018 contain the values 10 and 12, respectively. Determine the new final value of $t2.

Answer 3

(a) Loop Execution and Register Values

The loop executes 4 iterations.

Iteration	Loaded Value	$t2	$t0
1	5	5	1
2	8	13	2
3	3	16	3
4	6	22	4

Final register values:

$\$t0 = 4,\ \$t2 = 22,\ \$a0 = 0\text{x}10010010$

(b) Memory Write and Instruction Count

The final instruction is:

sw $t2, 0($a1)

Therefore:

- **Memory address: 0x10020000**
- **Stored value: 22**

The program executes 3 initialization instructions, 5 instructions per loop iteration, and 1 final store instruction.

$$IC = 3 + (5 \times 4) + 1 = 24$$

(c) Modified Memory Access

With the increment changed to 8 bytes, the program accesses:

Iteration	Memory Address	Loaded Value	Iteration
1	0x10010000	5	1
2	0x10010008	3	2
3	0x10010010	10	3
4	0x10010018	12	4

Therefore:

$$\$t2 = 5 + 3 + 10 + 12 = 30$$

Q4. C-to-MIPS Code Translation

Consider the following C program:

```
int A[5] = {3, 7, 2, 8, 4};

int sum = 0;



for (int i = 0; i < 5; i++)

    if (A[i] > 4)

        sum += A[i];
```

Assume:

- $s0 stores the base address of array A.
- $s1 stores the variable sum, initially 0.
- $t0 stores the loop variable i, initially 0.
- Each array element occupies 4 bytes.
- All array elements are signed 32-bit integers.
- Branch delay slots are not used.

Complete the following MIPS assembly code by filling in blanks (1)–(8). Each blank represents one instruction or operand.

```
        addi $t0, $zero, 0
LOOP:   slti $t1, $t0, (1)
        beq  $t1, $zero, END
        sll  $t2, $t0, (2)
        add  $t3, $s0, $t2
        (3)
        addi $t5, $zero, 4
        slt  $t6, $t5, $t4
        (4)
        (5)
SKIP:   addi $t0, $t0, (6)
        (7)
END:    (8)
```

(a) Fill in blanks (1)–(4) to implement the loop condition, array addressing, and conditional branch.

(b) Fill in blanks (5)–(8) to complete the program. For blank (8), use nop to mark the end of the program.

(c) Determine the final value stored in $s1 after executing the program.

Answer 4

(a) Blanks (1)–(4)

Blank	Answer	Purpose
(1)	5	Loop condition: \(i<5\)
(2)	2	Multiply array index by 4
(3)	lw $t4, 0($t3)	Load A[i]
(4)	beq $t6, $zero, SKIP	Skip addition if \(A[i]\leq4\)

(b) Blanks (5)–(8)

Blank	Answer	Purpose
(5)	add $s1, $s1, $t4	Update sum
(6)	1	Increment loop index
(7)	j LOOP	Repeat loop
(8)	nop	No operation

(c) Final Value

Only array elements greater than 4 are added:

$$A = \{3,7,2,8,4\} \quad sum = 7 + 8 = \boxed{15}$$

Therefore:

$$\boxed{\$s1 = 15}$$

Q5. Single-Cycle Datapath Analysis

Consider a standard MIPS single-cycle datapath supporting R-type, lw, sw, beq, and addi instructions. Assume the following initial register values:

Register	Value
$s0	100
$s1	20
$s2	5
$t0	0
$t1	0

The following instructions are executed using the initial register values.

```
I1: add  $t0, $s0, $s1

I2: lw   $t1, 8($s0)

I3: beq  $s1, $s2, TARGET
```

Assume Mem[108] = 75 and that all memory addresses are byte addresses.

(a) Complete the following two tables for instructions I1–I3. Some control signal values are provided as hints. Use X for don't-care values.

Instruction	**RegDst**	**ALUSrc**	**MemtoReg**	**RegWrite**
I1: add	**1**	?	?	**1**
I2: lw	?	**1**	?	?
I3: beq	?	?	**X**	?
Instruction	**MemRead**	**MemWrite**	**Branch**	**ALUOp**
I1: add	?	**0**	?	?
I2: lw	**1**	?	?	**00**
I3: beq	?	?	**1**	?

(b) For instructions I1 and I2, determine:

1. The two ALU input values.
2. The ALU result.
3. The destination register and the value written to it.

Answer 5

(a) Control Signal Analysis

Instruction	RegDst	ALUSrc	MemtoReg	RegWrite
I1: add	1	0	0	1
I2: lw	0	1	1	1
I3: beq	X	0	X	0
Instruction	MemRead	MemWrite	Branch	ALUOp
I1: add	0	0	0	10
I2: lw	1	0	0	00
I3: beq	0	0	1	01

(b) ALU and Register Analysis

I1: add $t0, $s0, $s1

ALU inputs:

$$[A = 100, \quad B = 20]$$

ALU result:

$$[100 + 20 = \boxed{120}]$$

Destination register and value:

$$[\boxed{\$t0 = 120}]$$

`I2: lw $t1, 8($s0)`

ALU inputs:

$$[A = 100, \quad B = 8]$$

Effective memory address:

$$[100 + 8 = \boxed{108}]$$

Since (Mem[108]=75), the destination register is: $[\boxed{\$t1 = 75}]$

Q6. Single-Cycle Datapath Performance

Consider a MIPS single-cycle processor supporting R-type (add, sub), Load (lw), Store (sw), and Branch (beq) instructions. The delays of the datapath components are given below.

Component	Delay (ps)
Instruction Memory	200
Register File Read	100
ALU	150
Data Memory	250
Register File Write	100
Sign Extension	50
Multiplexer (MUX)	30

Assume the following critical paths for each instruction type.

Instruction	Components on Critical Path
R-type	Instruction Memory → Register File Read → MUX → ALU → MUX → Register File Write
lw	Instruction Memory → Register File Read → MUX → ALU → Data Memory → MUX → Register File Write
sw	Instruction Memory → Register File Read → MUX → ALU → Data Memory
beq	Instruction Memory → Register File Read → ALU → MUX

For this exercise, calculate each path delay by summing the listed component delays. Ignore other delays and parallel execution effects.

(a) Datapath Execution Flow

Based on Table 2, briefly explain the function of each datapath component involved in executing the following three instruction types:

1. R-type (add)
2. Load (lw)
3. Store (sw)

For each instruction, describe how data flows through the datapath and how the final result is handled.

(b) Critical Path and Clock Cycle Time

Calculate the critical path delay for each instruction type (R-type, lw, sw, and beq). Determine the minimum clock cycle time of the single-cycle processor and identify the instruction that determines it.

(c) Datapath Improvement and Speedup

Suppose the Data Memory delay is reduced from **250 ps to 150 ps**, while all other component delays remain unchanged.

Calculate:

1. The new minimum clock cycle time.
2. The overall processor speedup.

Answer 6

(a) Datapath Execution Flow

1. R-type (add)

- Instruction Memory: Fetches the instruction.
- Register File: Reads two source registers.
- MUX: Selects the second register value as the ALU input.
- ALU: Performs addition.
- MUX: Selects the ALU result for write-back.
- Register File: Writes the result into the destination register.

2. Load (lw)

- Instruction Memory: Fetches the instruction.
- Register File: Reads the base address register.
- MUX: Selects the sign-extended immediate as the ALU input.
- ALU: Calculates the effective memory address.
- Data Memory: Reads data from the calculated address.
- MUX: Selects the memory data for write-back.
- Register File: Writes the loaded value into the destination register.

3. Store (sw)

- Instruction Memory: Fetches the instruction.
- Register File: Reads the base address register and the register containing the data to store.
- MUX: Selects the sign-extended immediate as the ALU input.
- ALU: Calculates the effective memory address.
- Data Memory: Writes the register value to the calculated address.

(b) Critical Path and Clock Cycle Time

R-type:

$$[T_R = 200 + 100 + 30 + 150 + 30 + 100 = \boxed{610 \text{ ps}}]$$

Load (lw):

$$[T_{lw} = 200 + 100 + 30 + 150 + 250 + 30 + 100 = \boxed{860 \text{ ps}}]$$

Store (sw):

$$[T_{sw} = 200 + 100 + 30 + 150 + 250 = \boxed{730 \text{ ps}}]$$

Branch (beq):

$$[T_{beq} = 200 + 100 + 150 + 30 = \boxed{480 \text{ ps}}]$$

The minimum clock cycle time is determined by the longest instruction delay:

$$[T_{clock} = \max(610,860,730,480)]$$

$$[\boxed{T_{clock} = 860 \text{ ps}}]$$

Critical instruction: lw

(c) Datapath Improvement and Speedup

The Data Memory delay decreases from 250 ps to 150 ps.

Updated Instruction Delays

Instruction	Original Delay	New Delay
R-type	610 ps	610 ps
lw	860 ps	760 ps
sw	730 ps	630 ps
beq	480 ps	480 ps

1. New Minimum Clock Cycle Time

$$\left[T_{clock,new} = \max(610,760,630,480)\right]$$

$$\left[\boxed{T_{clock,new} = 760 \text{ ps}}\right]$$

2. Overall Processor Speedup

Assuming the same instruction count and one clock cycle per instruction:

$$\left[Speedup = \frac{T_{old}}{T_{new}} = \frac{860}{760}\right]$$

$$\left[\boxed{Speedup \approx 1.132}\right]$$

The processor achieves approximately 13.2% higher performance.

Q7. Pipeline Timing and Performance

Consider a processor with a standard five-stage MIPS pipeline:

$$[IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB]$$

The processor executes the following instructions:

```
I1: lw   $t0, 0($s0)

I2: add  $t1, $s1, $s2

I3: sub  $t2, $s3, $s4

I4: sw   $t1, 4($s0)

I5: and  $t3, $t4, $t5
```

Assume that each stage takes one clock cycle, the pipeline is initially empty, and instructions are issued in program order without stalls. For timing purposes, every instruction passes through all five stages.

(a) Pipeline Stage Analysis

For Cycle 3 and Cycle 5, identify which instruction occupies each pipeline stage (IF, ID, EX, MEM, and WB). Write "None" if a stage is empty.

(b) Pipeline Performance

Assume each pipeline stage takes 200 ps, and ignore pipeline register overhead.

Calculate:

1. The total execution time for five instructions using a single-cycle processor.
2. The total execution time using a five-stage pipelined processor.
3. The speedup achieved by pipelining.

Answer 7

(a) Pipeline Stage Analysis

Pipeline Stage	Cycle 3	Cycle 5
IF	I3 (sub)	I5 (and)
ID	I2 (add)	I4 (sw)
EX	I1 (lw)	I3 (sub)
MEM	None	I2 (add)
WB	None	I1 (lw)

(b) Pipeline Performance

1. Single-Cycle Processor

$$[T_{single} = 5 \times 200 = 1000 \text{ ps}]$$

$$[T_{total,single} = 5 \times 1000 = \boxed{5000 \text{ ps}}]$$

2. Pipelined Processor

$$[Cycles = k + n - 1 = 5 + 5 - 1 = 9]$$

$$[T_{total,pipeline} = 9 \times 200 = \boxed{1800 \text{ ps}}]$$

3. Pipeline Speedup

$$\left[Speedup = \frac{5000}{1800} = \boxed{2.78}\right]$$

Q8. Pipelined Datapath and Control Signals

Consider a standard five-stage MIPS pipelined processor:

$$[IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB]$$

The processor supports R-type, lw, sw, and beq instructions.

The control signals are generated during the ID stage and passed through the appropriate pipeline registers.

Consider the following instructions:

```
I1: lw   $t0, 8($s0)

I2: add  $t1, $s1, $s2

I3: sw   $t1, 12($s0)
```

Assume that the pipeline operates without stalls or hazards, and that each instruction advances one stage per clock cycle.

(a) Pipeline Control Signal Classification

Complete the following table by identifying the pipeline stage in which each control signal is used.

Use **EX, MEM, or WB**.

Control Signal	**Pipeline Stage**
RegDst	?
ALUSrc	EX
ALUOp	?
Branch	?
MemRead	?
MemWrite	MEM
RegWrite	?
MemtoReg	?

(b) Pipeline Register and Control Signal Analysis

For instruction I1 (lw $t0, 8($s0)), answer the following questions:

1.Which pipeline register carries the control signals from ID to EX?

2.Which pipeline register carries the memory control signals from EX to MEM?

3.Which pipeline register carries the write-back control signals from MEM to WB?

For each pipeline register, identify the relevant control signals and their values for the lw instruction.

Answer 8

(a) Pipeline Control Signal Classification

Control Signal	Pipeline Stage
RegDst	EX
ALUSrc	EX
ALUOp	EX
Branch	MEM
MemRead	MEM
MemWrite	MEM
RegWrite	WB
MemtoReg	WB

(b) Pipeline Register and Control Signal Analysis

1. ID → EX: ID/EX Pipeline Register

ID/EX register carries the control signals required by EX, MEM, and WB stages.

For lw:

- EX: RegDst = 0, ALUSrc = 1, ALUOp = 00
- MEM: Branch = 0, MemRead = 1, MemWrite = 0
- WB: RegWrite = 1, MemtoReg = 1

2. EX → MEM: EX/MEM Pipeline Register

EX/MEM register carries the control signals required by the MEM and WB stages.

For lw:

- MEM: Branch = 0, MemRead = 1, MemWrite = 0
- WB: RegWrite = 1, MemtoReg = 1

3. MEM → WB: MEM/WB Pipeline Register

MEM/WB register carries the control signals required by the WB stage.

For lw:

- WB: RegWrite = 1, MemtoReg = 1